

Guided Practice: First Node and Express Project

Outcome

You will create a Node and Express web application that uses and runs on the MEAN stack.

Resources Needed

- VCASTLE
- OR --
- System configured for MEAN stack development.

Level of Difficulty

Moderate

Deliverables

Deliverables are marked in red font or with a red picture border around the screenshot. **Your name should be visible as directed in screenshots that you submit.**

You will submit a link to your GitHub repository that contains files associated with this assignment. This will include:

1. Screenshot of the application running.
2. Committed code files.

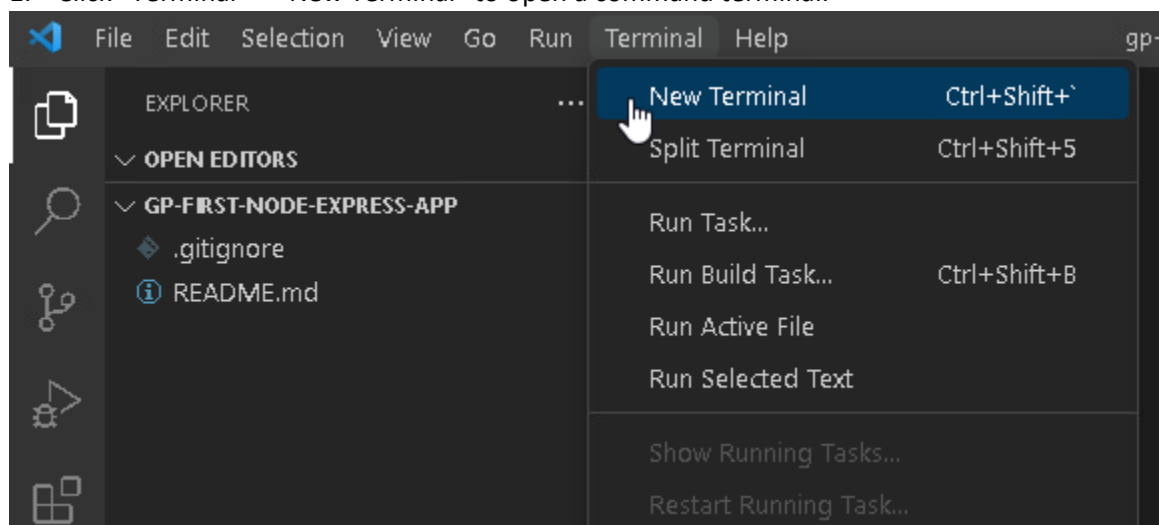
Program Specifications

This practice program is designed to demonstrate and help you gain a deeper understanding of creating and running a web application using Node and Express.

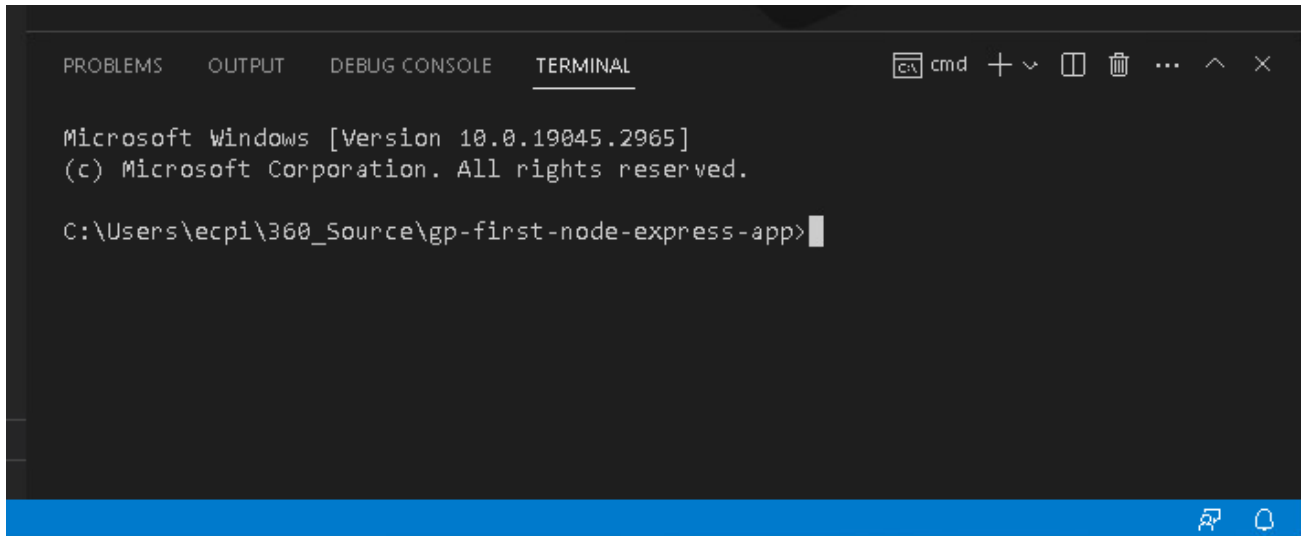
Development Steps

Open a Terminal Window

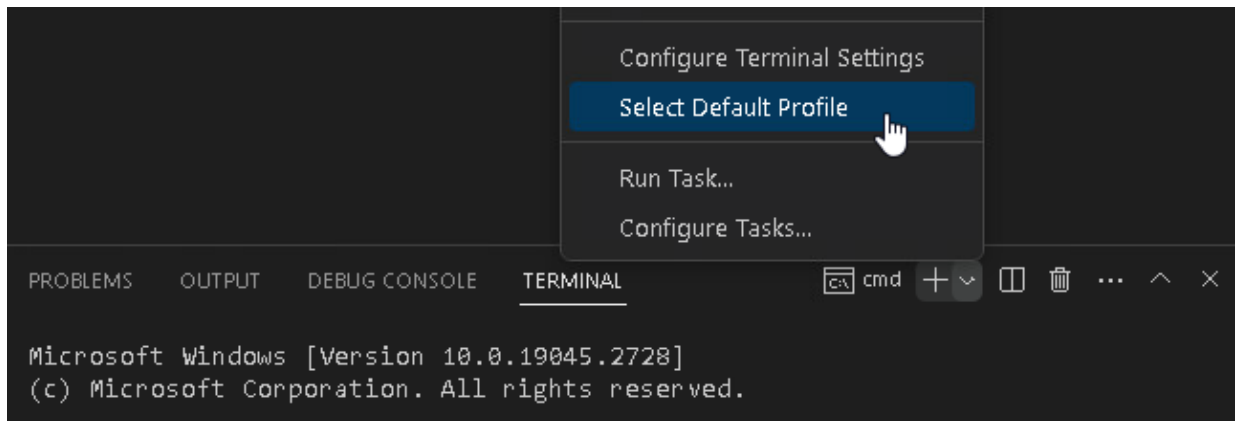
1. Click “Terminal” > “New Terminal” to open a command terminal.



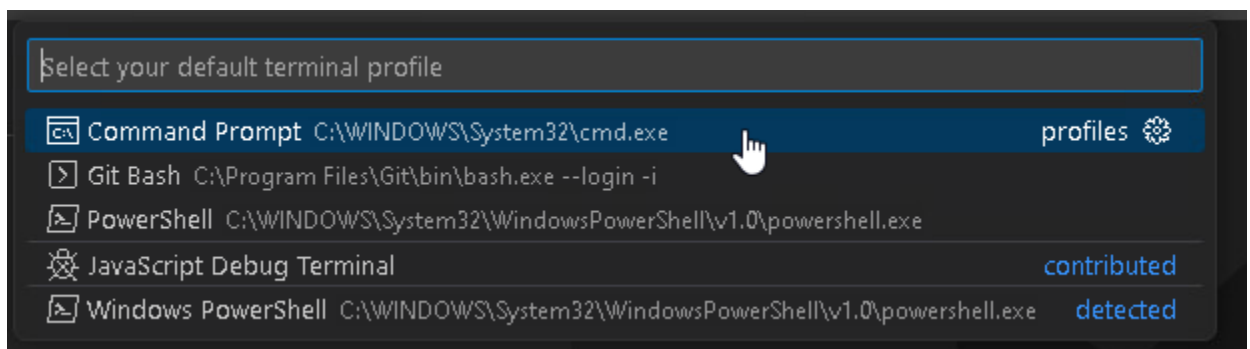
2. This will open a terminal window in the lower part of VS Code with the base folder (your repo folder) as the default.



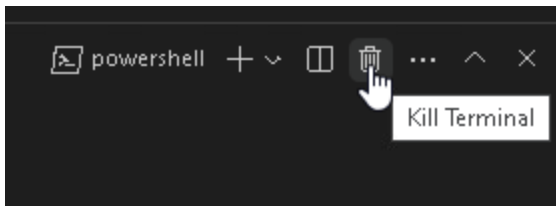
- a. If the terminal opens as a Windows command prompt (as shown above), proceed to the next section (Create the Project); if it opens as a PowerShell prompt or other terminal type, change the profile to use a command prompt by selecting the down arrow next to the + symbol in the terminal and selecting Select Default Profile



- b. In Profile Selection, select Command Prompt



- c. Kill the existing terminal by clicking the Kill Terminal icon, then go back to step 2 to open a Windows Command Terminal.



Create & Run the Application:

1. In the terminal window opened in the previous section, type the command:
`express first_application --view=ejs`
2. Press enter. This will create a directory with the required dependencies for the express application. Output should look similar to the following:

```
create : first_application\  
create : first_application\public\  
create : first_application\public\javascripts\  
create : first_application\public\images\  
create : first_application\public\stylesheets\  
create : first_application\public\stylesheets\style.css  
create : first_application\routes\  
create : first_application\routes\index.js  
create : first_application\routes\users.js  
create : first_application\views\  
create : first_application\views\error.ejs  
create : first_application\views\index.ejs  
create : first_application\app.js  
create : first_application\package.json  
create : first_application\bin\  
create : first_application\bin\www  
  
change directory:  
  > cd first_application  
  
install dependencies:  
  > npm install  
  
run the app:  
  > SET DEBUG=first-application:* & npm start
```

3. The output gives us the steps to prepare and run the application:
 - a. Change directory (cd) into the application directory (folder).
Type the command:
`cd first_application`
Press enter.

```

change directory:
  > cd first_application

install dependencies:
  > npm install

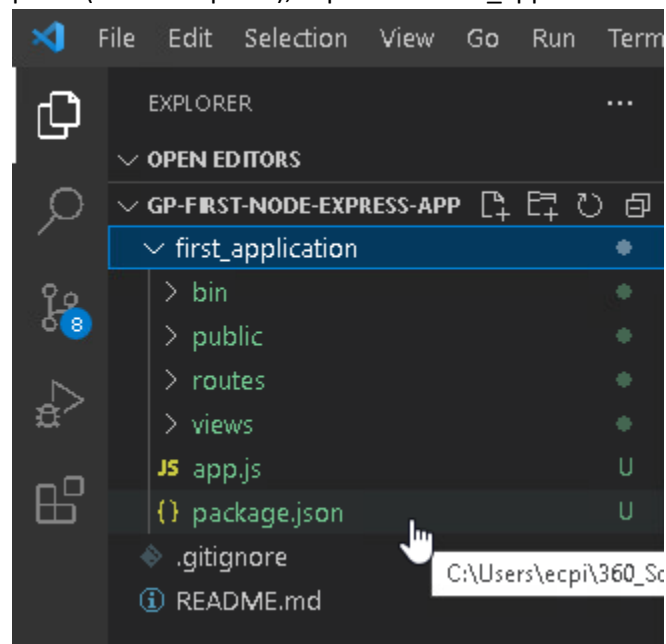
run the app:
  > SET DEBUG=first-application:* & npm start

C:\Users\ecpi\360_Source\gp-first-node-express-app>cd first_application
C:\Users\ecpi\360_Source\gp-first-node-express-app\first_application>

```

- b. Install the application's needed dependencies.

The application's dependencies are defined in the file named "package.json". Let's take a look at that file to see what it contains before we install the dependencies. In the VS Code Explorer panel (left-hand panel), expand the first_application folder and click on package.json:



This opens the package.json file. The contents should appear as follows:

```
{ } package.json U ×
first_application > { } package.json > ...
1  {
2    "name": "first-application",
3    "version": "0.0.0",
4    "private": true,
5    "scripts": {
6      "start": "node ./bin/www"
7    },
8    "dependencies": {
9      "cookie-parser": "~1.4.4",
10     "debug": "~2.6.9",
11     "ejs": "~2.6.1",
12     "express": "~4.16.1",
13     "http-errors": "~1.6.3",
14     "morgan": "~1.9.1"
15   }
16 }
17
```

The contents define the libraries and their versions needed to run the application. The needed libraries will be pulled and stored in a folder called "node_modules". Note that this folder will contain A LOT of files - which is why it is not included in a commit (it's in the .gitignore for Node applications). Any time the package.json file is modified, the dependencies need to be updated.

To create or update the dependencies, run the command:

npm install

```
C:\Users\ecpi\360_Source\gp-first-node-express-app\first_application>npm install
added 54 packages, and audited 55 packages in 7s

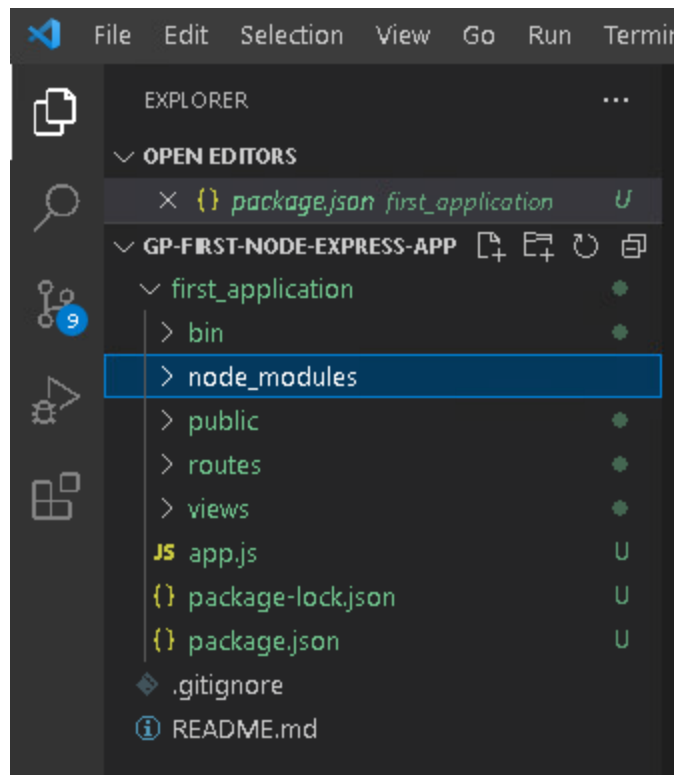
4 vulnerabilities (3 high, 1 critical)

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.

C:\Users\ecpi\360_Source\gp-first-node-express-app\first_application>
```

Do not run the audit fix - that will change package.json and may cause version conflicts. As these are not production applications that we'll only run locally, the vulnerabilities are not a concern. Note that following the running of npm install the node_modules folder was created:



You may note that it appears in grey text and doesn't have the "stop light" next to the folder name as is seen on other folders. This is because this folder is NOT included in source_control (and we don't want it to be). To give an idea, even for this very simple application with a relatively small number of dependencies, the node_modules folder contains 75 folders and 352 files. We NEVER modify the contents of the node_modules folder directly. If you think something is wrong with the dependencies, delete the folder and re-run npm install.

c. Run the app.

To run the application, type the following command and press enter (note that this is the command provided when the app was created):

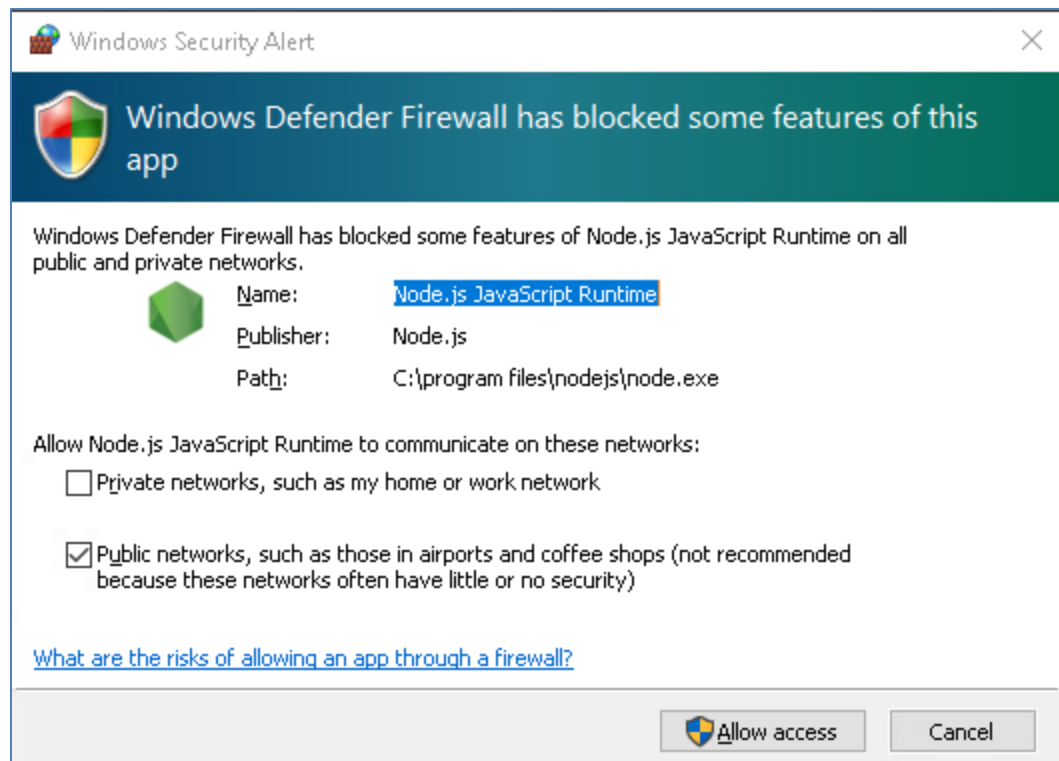
```
SET DEBUG=first-application:* & npm start
```

```
C:\Users\ecpi\360_Source\gp-first-node-express-app\first_application>SET DEBUG=first-application:* & npm start

> first-application@0.0.0 start
> node ./bin/www

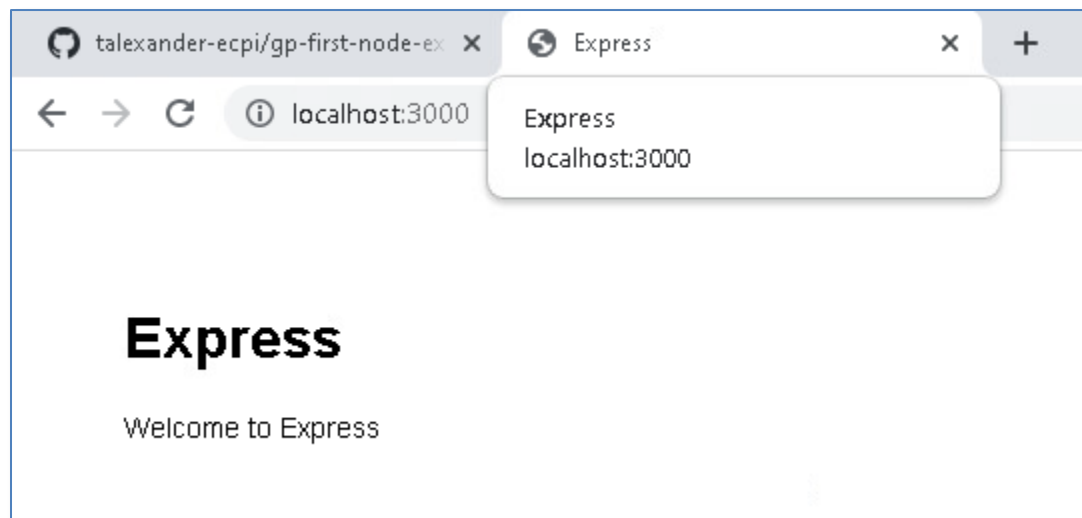
first-application:server Listening on port 3000 +0ms
```

If you get a Windows Defender Security Alert, select "Allow Access" to proceed.



To see the application running, open a browser window and type the address:
<http://localhost:3000>

The running application should appear as follows:

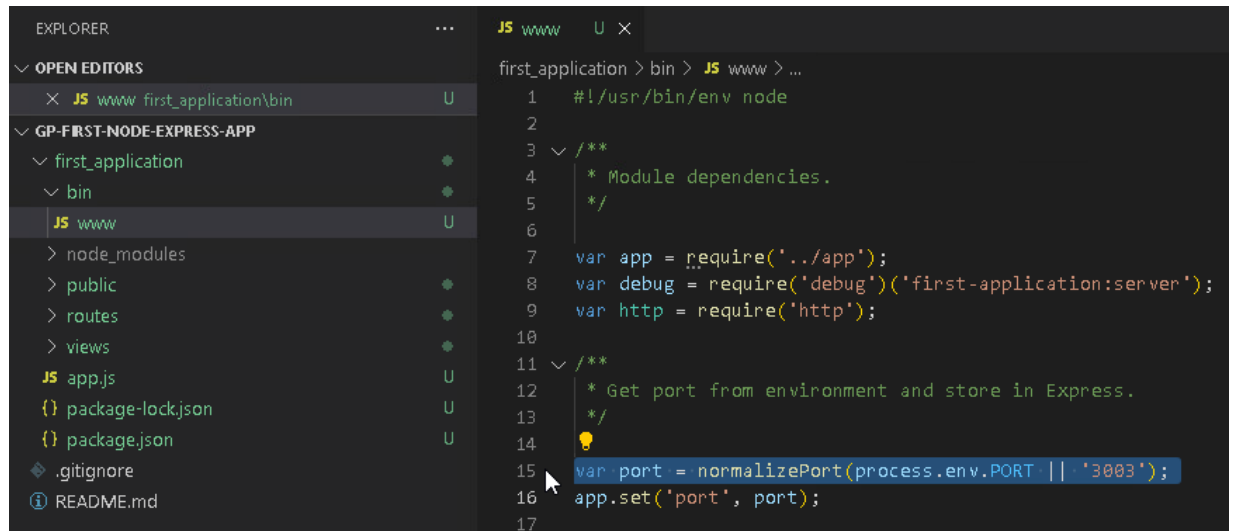


Close the browser window. In the VS Code terminal press the CTRL and C keys (CTRL+C) simultaneously to terminate the application server. You should almost always do this when making changes to application code.

Review and Modify the Application Code:

1. First, we'll change the port that the application runs on. It is handy to know how to do this in case we are using the default port (3000) for something else. After doing this, if you want to run the application you would need to open the browser to <http://localhost:3003>.
 - a. Expand the "bin" folder.

- b. Open the “www” file.
- c. Find the line that defines the port. In the example below it is line 15 (highlighted). Change the “3000” to “3003”.



```
EXPLORER
  OPEN EDITORS
    JS www first_application\bin
  GP-FIRST-NODE-EXPRESS-APP
    first_application
      bin
        JS www
      node_modules
      public
      routes
      views
      app.js
      package-lock.json
      package.json
      .gitignore
      README.md

JS www
  first_application > bin > JS www > ...
  1  #!/usr/bin/env node
  2
  3  /**
  4   * Module dependencies.
  5   */
  6
  7  var app = require('../app');
  8  var debug = require('debug')('first-application:server');
  9  var http = require('http');
  10
  11  /**
  12   * Get port from environment and store in Express.
  13   */
  14
  15  var port = normalizePort(process.env.PORT || '3003');
  16  app.set('port', port);
  17
```

2. Next, we will change how we handle the incoming get request and form a response.
 - a. Expand the routes folder.
 - b. Open the index.js file.
 - c. Create a list of favorite places by replacing the code beneath the comment `/* GET home page */` as shown in the code below. Please enter places of your own choosing.

Note the creating of the myname variable as “plain old javascript”, and the assignment of the properties name and rating in each item in the list.

Also note the `res.render` function call. This call will pass the title property, the places property (assigned to the places array we created) and the myname property assigned to the myname variable we created. If you hover over each one in Visual Studio Code, it will tell you what the property is and what the local variable is!

Here is the updated code for index.js - make sure to incorporate you own favorites list and replace “Your Name” with your actual name:

```
var express = require('express');
var router = express.Router();

/* GET home page. */
router.get('/', function (req, res, next) {
  var places = [
    { name: 'Home', rating: 10 },
    { name: 'Grandmas House', rating: 9 },
    { name: 'San Francisco', rating: 5 },
    { name: 'San Antonio', rating: 6 },
    { name: 'Virginia Beach', rating: 5 }
  ];

  // create a name
```



```

var myname = "Your Name";

res.render('index', {
  title: 'Favorite Places',
  places: places,
  myname: myname
});
});

module.exports = router;

```

3. Next, we will modify the index.ejs file, which contains the html code and scriptlets (special tags used to present the data) used to create the view (what you see in the browser).
 - a. Expand the views folder.
 - b. Open the index.ejs file.
 - c. Modify the code to add processing and support of the data we created in index.js above.

Take note of the use of scriptlet tags to run the for loop and to access the data we have passed in. Of particular interest is the foreach used to process each item in the places list. Notice the closing %> tag at the end of the forEach, which allows for the tag to be placed in the html code.

Here is the updated code for index.ejs:

```

<!DOCTYPE html>
<html>
<head>
  <title>
    <%= title %>
  </title>
  <link rel='stylesheet' href='/stylesheets/style.css' />
</head>

<body>
  <h2>
    <%= myname %>
  </h2>
  <h2>List of Places I'd like to Visit and Rating 1 to 10</h2>

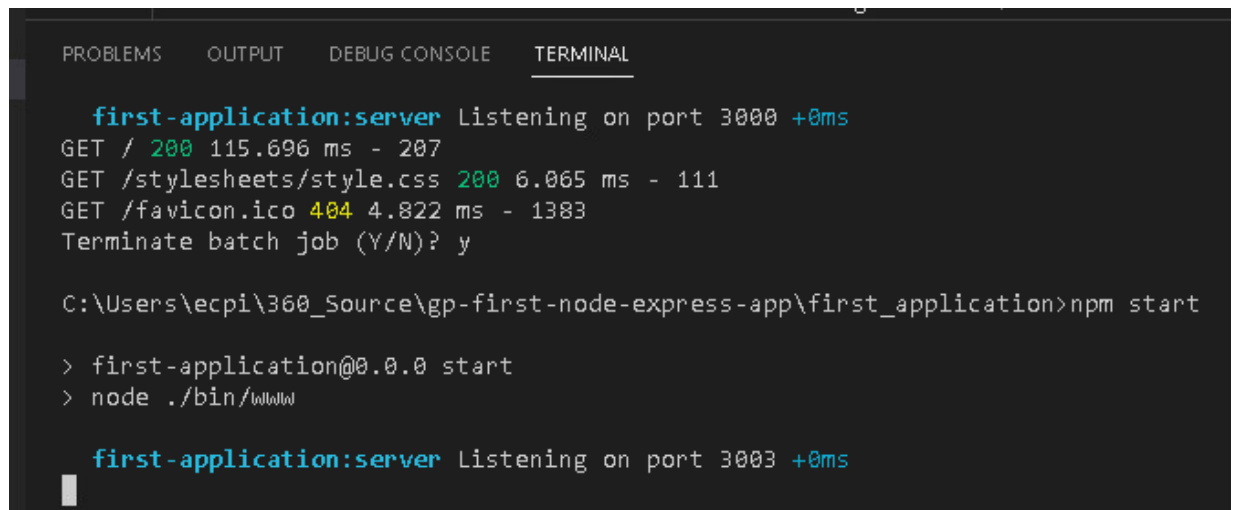
  <ul>
    <% places.forEach(function(place) { %>
      <li>
        <%= place.name %> : <%= place.rating %>
      </li>
    <% }); %>
  </ul>
</body>
</html>

```

4. Finally, we run the application to see the results of our changes.

a. In the terminal window, run the command:

npm start



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

first-application:server Listening on port 3000 +0ms
GET / 200 115.696 ms - 207
GET /stylesheets/style.css 200 6.065 ms - 111
GET /favicon.ico 404 4.822 ms - 1383
Terminate batch job (Y/N)? y

C:\Users\ecpi\360_Source\gp-first-node-express-app\first_application>npm start

> first-application@0.0.0 start
> node ./bin/www

first-application:server Listening on port 3003 +0ms
```

b. Open a browser to <http://localhost:3003> and view the fruits of your labor!

c. Take a screen capture of the running application and save it in the same folder as the README.md file.

Prepare to submit the code:

Take a screenshot that resembles the one below with the red border and save it in the same folder as your README.md file in the repository. Your name should be visible in the screenshot.

